

Security Audit

cNGN (Stablecoin)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	10
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Conclusion	50
Our Methodology	51
Disclaimers	53
About Hashlock	54

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The cNGN team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

cNGN is a regulated stablecoin infrastructure project designed to bring the Nigerian Naira on-chain through a fully collateralized, 1:1 Naira-backed digital asset. The platform provides secure and compliant blockchain-based payment rails for individuals, businesses, exchanges, and financial institutions, enabling fast, low-cost settlements, cross-border payments, asset management, and developer integrations via APIs and SDKs. The project emphasizes regulatory compliance, transparent proof-of-reserves, and interoperability across multiple blockchain networks to support the growth of Nigeria's digital financial ecosystem.

Project Name: cNGN

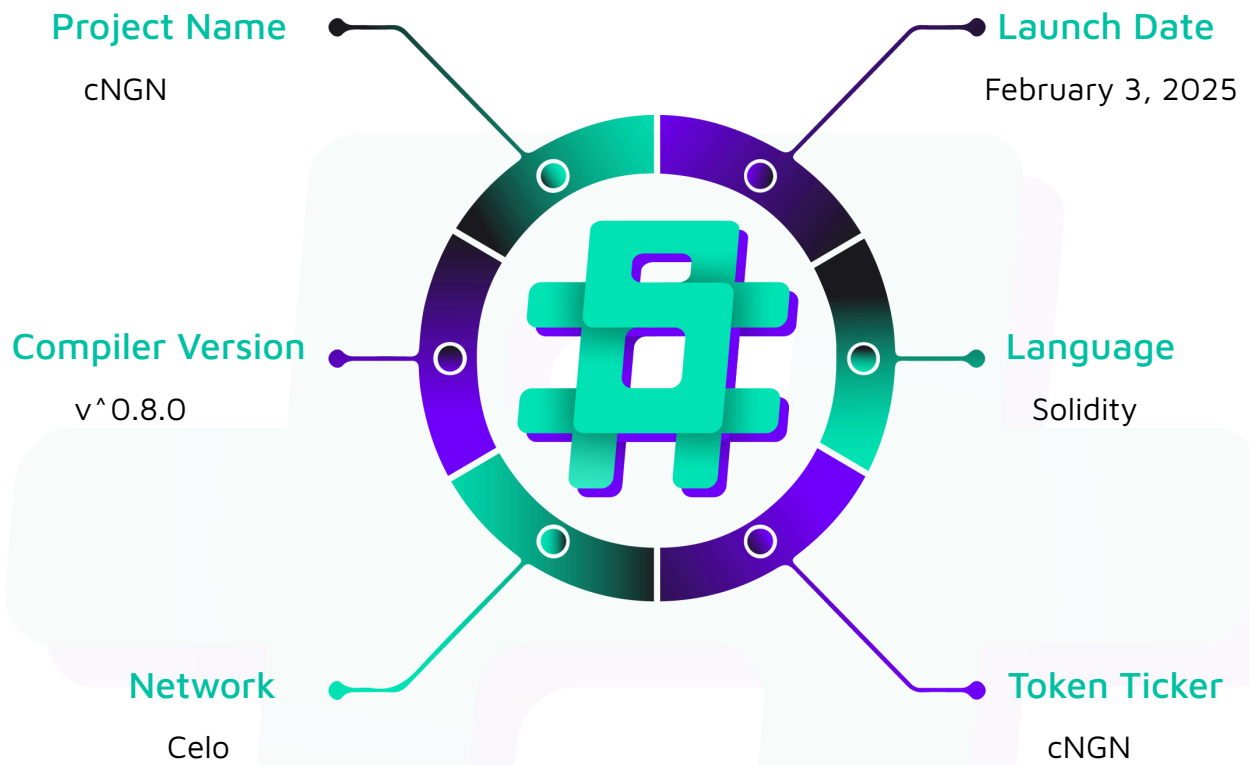
Project Type: Stablecoin

Compiler Version: ^0.8.0

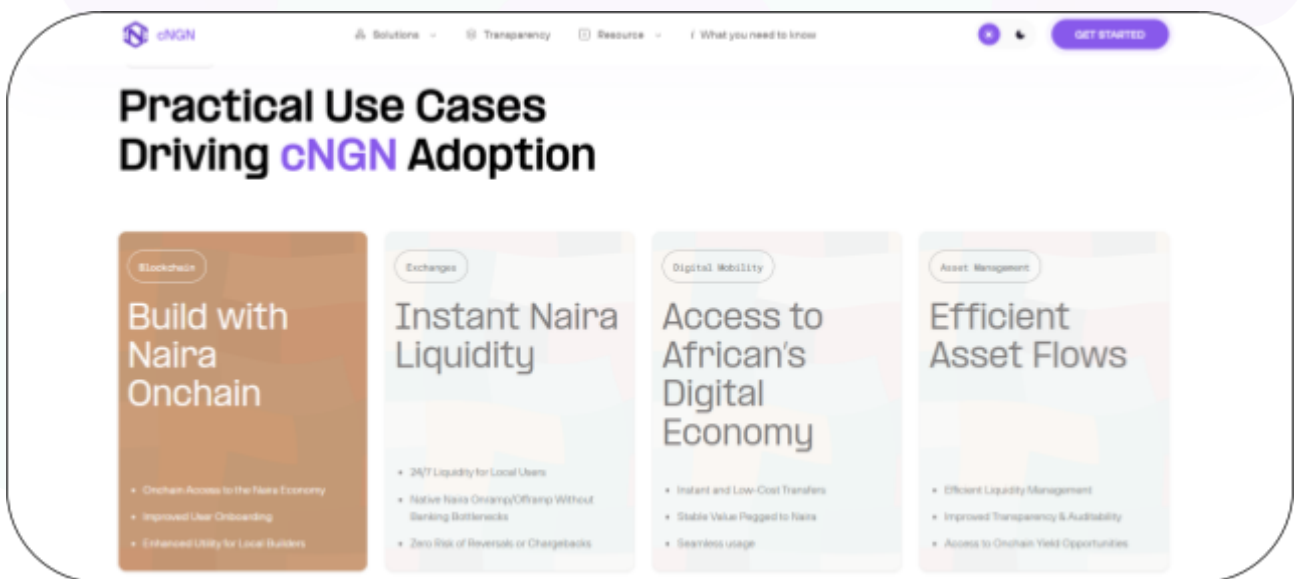
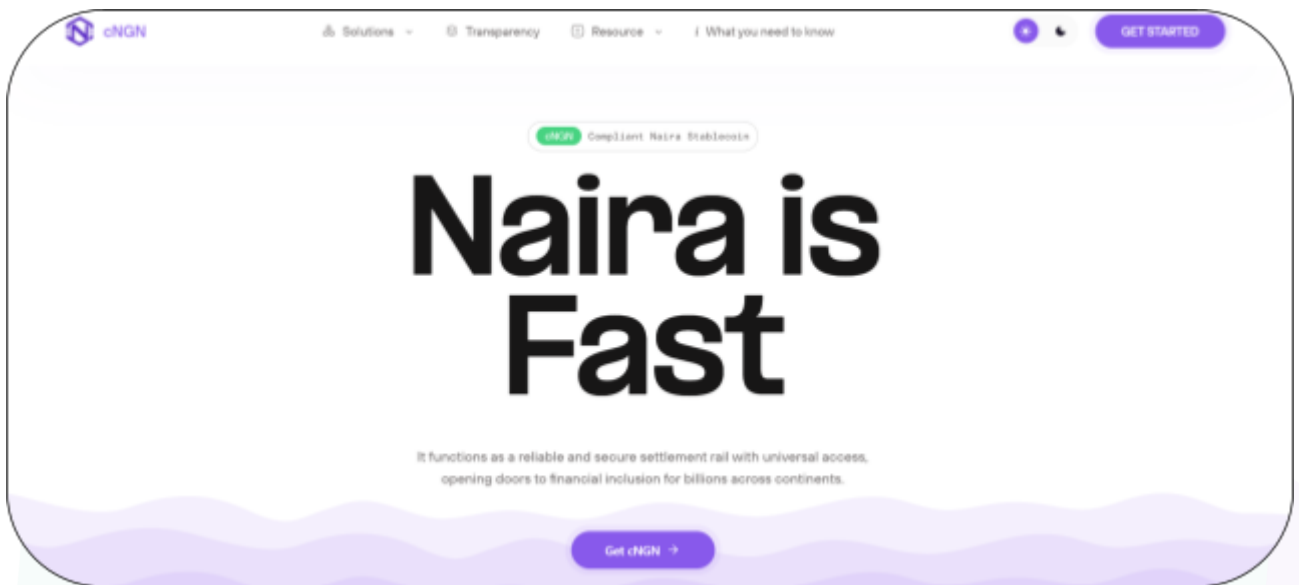
Website: <https://cngn.co/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the cNGN project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	cNGN Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	July, 2026
Contract 1	Cngn3.sol
Contract 2	Operations2.sol
Contract 3	Forwarder.sol
Audited GitHub Commit Hash	4c8f343bff6c468a5f112d3e470c29ae189852ea

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 Medium severity vulnerabilities

5 Low severity vulnerabilities

13 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Cngn3.sol - ERC-20 stablecoin (cNGN), 6 decimals, pegged to Naira. - Users can transfer, approve, and burn their own tokens. - Pre-approved minters can mint an exact, one-time authorized amount to any address. - Blacklisted addresses are blocked from transferring, minting, or burning. - Transfers from an approved "external sender" to an approved "internal user" are auto-redeemed (credited then burned) to model off-chain cash-out. - Supports gasless meta-transactions via one trusted forwarder. - Owner can pause all token activity and seize (burn) a blacklisted address's balance. - Delegates blacklist/whitelist/mint-authorization decisions to an external Operations2 contract.	Contract achieves this functionality.
Operations2.sol - Central permissions registry for the cNGN ecosystem. - Owner manages the blacklist. - Owner (or a trusted contract) authorizes minters and sets their exact mint amount.	

- Owner manages the external-sender / internal-user whitelists used for redemption. - Owner manages which addresses may relay meta-transactions. - Owner manages which contracts are trusted to act on its behalf. - Owner can pause/unpause its own administrative functions.	
Forwarder.sol - Enables gasless meta-transactions for cNGN (ERC-2771 style). - Users sign a request off-chain; owner or an authorized bridge relays it on-chain. - Verifies EIP-712 signatures and enforces sequential nonces to prevent replay. - Forwards the call to the target, appending the real signer's address. - Owner manages authorized bridge relayers. - Owner can pause/unpause relaying.	

Code Quality

This audit scope involves the smart contracts of the cNGN project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the cNGN project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Forwarder#execute/executeByBridge - Native ETH value is forwarded from the contract's pooled balance without validating it against the relayer's `msg.value`

Description

`execute()` and `executeByBridge()` are payable, and `_executeTransaction` forwards `req.value` to `req.to` via a low-level `.call{value: req.value}`. Nothing anywhere checks `msg.value == req.value`. `.call{value: ...}` spends from the Forwarder contract's own balance, not from the specific `msg.value` supplied in that transaction. Combined with a completely unguarded `receive() external payable {}` and no owner-only sweep/withdraw function anywhere in the contract, any ETH that ends up in the Forwarder (via `receive()`, or via an owner/bridge over-paying `msg.value` relative to `req.value` on some other request) becomes a shared, unattributed pool that any `canForward`-whitelisted (ordinary, non-admin) signer can route to themselves.

Vulnerability Details

Forwarder.sol:

- `execute` (lines 162–174) — `onlyOwner`, payable.
- `executeByBridge` (lines 177–189) — `onlyAuthorizedBridge`, payable.
- `_executeTransaction` (lines 112–159) — line 152–155: `req.to.call{gas: req.gas, value: req.value}(...)`. No comparison to `msg.value` anywhere in the function or its callers.
- `receive() external payable {}` (line 199) — accepts ETH from anyone, unconditionally.
- No `withdraw/rescue/sweep` function exists anywhere in the contract — the only way ETH ever leaves the contract is through `_executeTransaction`'s forwarding call.

Proof of Concept

Conceptual scenario:

1. ETH accumulates in the Forwarder contract — either through the unguarded `receive()` (anyone can send ETH by mistake or design), or because the owner calls `execute{value: X}(req)` where `req.value < X` (the excess is never refunded).
2. `Attacker` is an ordinary address that has been granted `canForward(Attacker) = true` in `Admin2` — a routine, non-admin permission used for normal gasless-transaction relaying, not a privileged/trusted role.
3. `Attacker` signs a fresh `ForwardRequest{ from: Attacker, to: Attacker, value: <pool balance>, gas: 21000, nonce: getNonce(Attacker), data: "" }`.
4. The owner or any authorized bridge — acting as an automated relayer that simply verifies signature + nonce + whitelist status before forwarding, per the system's documented "relayer submits the signed request" flow — calls `executeByBridge(req, sig)` with `msg.value = 0`.
5. `verify()` passes (`Attacker` legitimately signed their own request), `canForward/blacklist` checks pass, replay/nonce bookkeeping proceeds normally, and `req.to.call{value: <pool balance>}("")` pays the entire pooled ETH balance to `Attacker` — funds that were never contributed by `Attacker` and are unrelated to whoever originally sent them.

Impact

Total loss of any ETH balance the Forwarder contract accumulates, drainable by any `canForward`-whitelisted address with no special privilege required, and with no recovery function available to the owner even if caught in advance.

Recommendation

Require `msg.value == req.value` in both `execute` and `executeByBridge` before calling `_executeTransaction` (reverting otherwise), and add an owner-only rescue function to deliberately recover any ETH that legitimately needs to be pulled out (e.g., accidental direct sends via `receive()`).

Status

Resolved

[M-02] Cngn3#destroyBlackFunds - Reverts while the contract is paused, defeating its purpose as an incident-response tool

Description

`destroyBlackFunds` is deliberately declared without `whenNotPaused` (unlike every other state-changing entrypoint — `transfer`, `transferFrom`, `mint`, `burnByUser` all explicitly carry it), signaling it is meant to remain usable during an emergency pause to seize blacklisted funds. However, it calls `_burn`, which unconditionally calls `_beforeTokenTransfer`, and that internal hook itself carries `whenNotPaused`. As a result, `destroyBlackFunds` always reverts while the contract is paused— exactly the state an operator is most likely to be in when they need to use it.

Vulnerability Details

```
// Cngn3.sol
function destroyBlackFunds(address _blackListedUser) public virtual onlyOwner
nonReentrant returns (bool) {
    // no whenNotPaused here - appears intentional
    ...
    _burn(_blackListedUser, dirtyFunds);
    ...
}

function _burn(address account, uint256 amount) internal virtual {
    ...
    _beforeTokenTransfer(account, address(0), amount);
    ...
}

function _beforeTokenTransfer(address from, address to, uint256 amount) internal
virtual whenNotPaused {}
```

Impact

The emergency fund-seizure mechanism is unusable exactly when it is needed most (mid-incident, contract paused), forcing operators to trade off re-exposing the system to further exploitation in order to execute a seizure.

Recommendation

Remove `whenNotPaused` from `_beforeTokenTransfer` (it is redundant everywhere else, since every public entrypoint already enforces `whenNotPaused` independently), or split the burn logic so `destroyBlackFunds` uses a pause-independent internal burn path.

The same pattern also exists on the `Operations2` (`Admin2`) side: `addBlackList`, `removeCanMint`, `removeMintAmount`, and `blacklistExternalSender` all carry `whenNotPaused`. Pausing `Admin2` during an incident simultaneously disables the ability to blacklist an attacker or revoke a compromised minter — the exact levers an emergency pause is meant to enable, not disable. Consider exempting containment-only functions (`addBlackList`, `removeCanMint`, `removeMintAmount`) from the pause gate on both contracts.

Status

Resolved

Low

[L-01] Forwarder#_executeTransaction - Signer's nonce is consumed even when the relayed call fails

Description

The on-chain nonce is bumped and the replay hash is recorded before the outcome of the low-level `.call` to `req.to` is known, and `success` is never `require'd` — a forwarded request whose target call fails for any reason (target paused, blacklist changed between signing and relay, target-side revert, insufficient headroom) still permanently consumes the signer's nonce. The signer must re-sign an entirely new request to retry, since sequential nonce matching in `verify()` leaves no cancel/skip/retry path.

Vulnerability Details

```
_nonces[req.from] = req.nonce + 1;    // committed unconditionally
emit NonceIncremented(req.from, _nonces[req.from]);
...
(bool success, bytes memory returndata) = req.to.call{gas: req.gas, value:
req.value}(...);
emit Executed(_msgSender(), success, returndata);
return (success, returndata);    // failure is never reverted
```

Impact

A relayer (whether through bad timing, a compromised bridge, or simple bad luck with concurrent state changes) can cause a user's signed request to be permanently invalidated without the intended action ever taking effect — an availability/griefing issue against the signer, not the relayer.

Recommendation

Either revert the entire transaction on `success == false` (so the nonce bump and replay marking are rolled back together with the failed call), or provide an explicit, signer-controlled cancellation mechanism independent of failed relay attempts.

Status

Resolved

[L-02] Cngn3 / Forwarder - No path to update adminOperationsContract after deployment

Description

Both `Cngn3.updateAdminOperationsAddress` and `Forwarder`'s equivalent updater are fully commented out, and `Forwarder.adminOperationsContract` is declared `immutable`. `Cngn3.initialize()` also performs no zero-address validation on either `_trustedForwarderContract` or `_adminOperationsContract`, unlike the live `updateForwarderContract`, which does validate. If the Admin2/Operations registry ever needs replacing (e.g., a bug fix, or a planned migration), neither `Cngn3` nor `Forwarder` has any code path to repoint at a new instance short of a full logic-contract upgrade.

****Carried over from prior audits, still unfixed:**** this exact gap (missing zero-address validation on `adminOperationsContract/trustedForwarderContract`) was already flagged against the earlier `cngn.sol/forwarder.sol` by CertiK (finding CON-04, v1 report) — the team's response at the time was to "acknowledge... and opt to make no change." It was raised again against `Cngn2.sol` (CSU-11, v2 report), where the team said they "added the check to functions that are necessary," implying `initialize()/constructor` paths were deliberately left unchecked. The current `Cngn3.sol/Forwarder.sol` inherits the same unaddressed gap — and it is now strictly worse in `Forwarder.sol`, where `adminOperationsContract` is `immutable`, so a bad deployment can never be corrected at all (not even via a governed setter, since none exists).

Vulnerability Details

```
// Cngn3.sol
// function updateAdminOperationsAddress(address _newAdmin) public virtual onlyOwner
returns (bool) { ... } // commented out

// Forwarder.sol
address public immutable adminOperationsContract;
// function updateAdminOperationsAddress(address newAdmin) external onlyOwner returns
(bool) { ... } // commented out
```

Impact

Forces a full token/forwarder logic redeployment (with all associated migration risk) to recover from any issue in the shared admin registry, rather than a simple, auditable admin-only pointer update.

Recommendation

Restore a guarded `updateAdminOperationsAddress` function (with zero-address checks, matching the pattern already used in `updateForwarderContract`) in both `Cngn3` and `Forwarder`, and add zero-address validation to `Cngn3.initialize()`.

Status

Resolved

[L-03] Cngn3 / Operations2 - Proxy initializers are not atomicity-protected against front-running

Description

`Cngn3.initialize()` and `Admin2.initialize()` are both public initializer with no caller restriction — standard for the proxy pattern, but only safe if the proxy's deployment transaction and its `initialize()` call happen atomically (e.g., in the same transaction via a factory, or via `ERC1967Proxy` constructed with init data). `Admin2.initialize()` in particular grants the caller both `canForward` and `canMint`, and implicitly ownership (via `__Ownable_init()`, which sets the deployer/initializer as owner). If any production deployment ever separates "deploy proxy" from "call initialize" into two transactions, a front-runner monitoring the mempool could call `initialize()` first and become owner, gaining the ability to self-arm `mintAmount` for unlimited minting.

Vulnerability Details

```
// Operations2.sol
function initialize() public initializer {
    __Ownable_init();           // caller becomes owner
    __ReentrancyGuard_init();
    __Pausable_init();
    canForward[_msgSender()] = true;
    canMint[_msgSender()] = true; // caller becomes a minter
}
```

The implementation contracts correctly call `_disableInitializers()` in their constructors, which protects the implementation itself — but does not protect a proxy whose constructor and `initialize()` call are not bundled into one transaction.

Impact

If deployment is ever non-atomic, a front-runner gains full ownership and unlimited minting rights over the entire protocol's admin registry — a total loss of protocol integrity. Impact is contingent on the deployment process, which is outside the in-scope files.

Recommendation

Verify and document that every production deployment constructs the proxy and calls `initialize()` in the same transaction (e.g., via `ERC1967Proxy(implementation, abi.encodeCall(Admin2.initialize, ()))` or an equivalent factory pattern). As defense-in-depth, consider having the deployer address itself call `initialize()` and immediately verify `owner() == expectedDeployer` on the same transaction/script, reverting the deployment otherwise.

Status

Acknowledged

[L-04] Cngn3#approve - No whenNotPaused or blacklist check, inconsistent with every other state-changing entrypoint

Description

Every other state-changing function in Cngn3.sol (transfer, transferFrom, mint, burnByUser) enforces both whenNotPaused and explicit blacklist checks via adminOperationsContract. approve()/_approve() enforce neither — they are copied verbatim from OpenZeppelin's base ERC20Upgradeable with no additional guards layered on top, unlike every other function in the contract. Confirmed by diffing Cngn3.sol against the vendored ERC20Upgradeable.sol: _transfer/_mint/_burn/_approve/_spendAllowance are otherwise identical to upstream, and transfer/transferFrom deviate from upstream specifically by adding pause/blacklist/redemption logic — approve is the one public state-changing entrypoint where that same treatment was never applied.

Vulnerability Details

```
function approve(address spender, uint256 amount) public virtual override returns
(bool) {
    address owner = _msgSender();
    _approve(owner, spender, amount);    // no whenNotPaused, no blacklist check
    return true;
}

function _approve(address owner, address spender, uint256 amount) internal virtual {
    require(owner != address(0), "ERC20: approve from the zero address");
    require(spender != address(0), "ERC20: approve to the zero address");
    _allowances[owner][spender] = amount;
    emit Approval(owner, spender, amount);
}
```

Impact

Low direct impact (approving does not itself move funds), but breaks the otherwise-consistent invariant that blacklisted/paused accounts cannot perform any

state-changing token action, and could interact unexpectedly with off-chain monitoring that assumes a blacklisted account is fully inert.

Recommendation

Add `whenNotPaused` and the same blacklist checks used elsewhere to `approve()` (and `_approve()` if it is ever called from another public entrypoint), for consistency with the rest of the contract's guard pattern.

Status

Acknowledged

[L-05] Cngn3#mint - Silently broken if only Operations2 (not Cngn3 itself) is paused

Description

`mint()`'s final step unconditionally calls `IAdmin(adminOperationsContract).removeCanMint(sender)` and requires it to return `true`. `Operations2.removeCanMint` carries its own independent `whenNotPaused` modifier gated on `Operations2`'s own pause flag — not `Cngn3`'s. Every other `Cngn3` entrypoint (`transfer`, `transferFrom`, `burnByUser`, `destroyBlackFunds`) only calls `Operations2` view getters (`isBlackListed`, `isInternalUserWhitelisted`, etc.), which carry no modifiers and are unaffected by `Operations2`'s pause state — `mint()` is the only `Cngn3` function whose success depends on a state-changing call into `Operations2`, making it uniquely coupled to `Operations2`'s pause flag even while `Cngn3` itself remains unpaused. This corrects the assumption in **[Q-04]** that pausing only `Operations2` "leaves `Cngn3` transfers and mints fully live" — that holds for transfers, but not for mints.

Vulnerability Details

```
// Cngn3.sol - mint()
_mint(_mintTo, _amount);
require(
    IAdmin(adminOperationsContract).removeCanMint(sender),
    "Failed to revoke minting authorization"
);

// Operations2.sol
function removeCanMint(address _User) public whenNotPaused onlyOwnerOrTrustedContract
returns (bool) { ... }
```

Impact

An operator pausing only `Operations2` for an unrelated administrative reason (e.g. to freeze blacklist changes during an incident) silently and completely disables minting on `Cngn3`, contrary to the apparent design intent of keeping the two pause domains independent.

Recommendation

Either don't hard-require the success of `removeCanMint` inside `mint()` (log/best-effort instead), or exempt `removeCanMint/removeMintAmount/addBlackList` from `Operations2`'s `whenNotPaused` gate — the latter also addresses the same containment-function concern already raised in `**[M-02]**`.

Status

Resolved

[L-06] Cngn3 / Forwarder - Full behavioral control is delegated to `adminOperationsContract`, a single externally-owned policy oracle

Description

Blacklist status, mint authorization, mint amounts, and both redemption whitelists are all resolved by an external call to `adminOperationsContract` (`IAdmin(...).isBlackListed/canMint/mintAmount/isInternalUserWhitelisted/isExternalSenderWhitelisted`). `Cngn3` and `Forwarder` hold no independent copy of this state and perform no sanity-checking on the values returned. If this contract is ever malicious, compromised, or misconfigured, minting, burning, and transfer eligibility for the entire token can be changed arbitrarily through it, with no override available in `Cngn3/Forwarder` themselves.

Vulnerability Details

Every mutating entrypoint in `Cngn3.sol` (`transfer`, `transferFrom`, `mint`, `burnByUser`, `destroyBlackFunds`) and every call in `Forwarder._executeTransaction` reads its authorization decision entirely from `adminOperationsContract`, e.g.:

```
require(!IAdmin(adminOperationsContract).isBlackListed(sender), "Signer is
blacklisted");
require(IAdmin(adminOperationsContract).canMint(sender), "Minter not authorized to
sign");
require(IAdmin(adminOperationsContract).mintAmount(sender) == _amount, "Attempting to
mint more than allowed");
```

This is compounded by **[L-02]**: there is no way to repoint `adminOperationsContract` at a replacement if the current one is ever found to be compromised or buggy — the two issues share the same root architectural dependency, viewed from different angles (L-02: can't be fixed if broken; this finding: what happens if it's malicious).

Impact

Full compromise of token integrity (supply manipulation, unauthorized mints, forced burns, bypassed blacklist/pause enforcement) if the external policy oracle is ever compromised or misconfigured. This is a design-level centralization/trust concentration risk rather than a standalone code bug — `Cngn3/Forwarder` correctly implement their

side of the contract, the risk is entirely in `adminOperationsContract`'s own integrity, which is outside these three files' scope.

Recommendation

Document this trust dependency explicitly for integrators and users. Consider a governance-controlled updater with a timelock for `adminOperationsContract` (partially addressed by fixing `**[L-02]**`), and ensure `Operations2/Admin2`'s own ownership is held by a multisig with appropriate operational safeguards (see `**[Q-07]**`).

Assessed at Medium rather than a higher severity: the practical attack precondition is that `adminOperationsContract` itself is already compromised or maliciously configured — at that point the compromise of the admin/oracle layer is the primary event, and this finding describes its downstream blast radius on `Cngn3/Forwarder` rather than an independently exploitable code defect in the three in-scope files.

Status

Acknowledged

QA

[Q-01] Cngn3 - Dead access-control modifiers left over from a real behavioral change between versions

Description

`onlyDeployerOrForwarder`, `onlyAdmin`, and `onlyTrustedForwarderCaller` are declared in `Cngn3.sol` but never applied to any function. `Cngn.sol` (V1) used `onlyDeployerOrForwarder` to gate `mint()/burnByUser()`; this guard was silently dropped starting at `Cngn2.sol`, so both functions are now fully public, relying only on the internal `canMint/blacklist` checks. This confirms a genuine access-control loosening across versions rather than dead code with no history — worth an explicit sign-off from the team that this was intentional.

Status

Acknowledged

[Q-02] Operations2 - Inconsistent authorization guard within the minter-lifecycle function family

Description

`addMintAmount` is `onlyOwner`, while its siblings `addCanMint`, `removeCanMint`, and `removeMintAmount` are all `onlyOwnerOrTrustedContract`. Not currently exploitable (no in-scope `trustedContract` calls `addMintAmount`), but the asymmetry is a latent trap if a future integration assumes a `trustedContract` has full lifecycle control over minters.

Note

Client's comment: `addMintAmount` remaining `onlyOwner` while its siblings are `onlyOwnerOrTrustedContract` is intentional, not an oversight: setting a specific mint allowance amount is treated as a higher-trust action than adding/removing minter status or clearing an amount, and we don't currently want a trusted contract able to self-authorize arbitrary mint ceilings. Leaving as-is; flagged for anyone integrating a new trusted contract that this asymmetry exists by design.

Status

Acknowledged

[Q-03] Operations2#removeCanMint - Redundant duplicate line

Description

```
function removeCanMint(address _User) public whenNotPaused onlyOwnerOrTrustedContract
returns (bool) {
    require(canMint[_User], "User is not a minter");
    canMint[_User] = false;
    mintAmount[_User] = 0;
    mintAmount[_User] = 0;    // duplicate, harmless but likely a copy-paste artifact
    ...
}
```

Status

Resolved

[Q-04] Cngn3 / Operations2 - Independent, non-propagating pause domains

Description

Cngn3 and Operations2 each inherit their own PausableUpgradeable and gate only their own functions; pausing one does not affect the other. An operator pausing only Operations2 (e.g. to freeze blacklist/minter management during an incident) leaves Cngn3 transfers fully live, and vice versa. Not directly attacker-triggerable, but a plausible operational trap during incident response worth documenting or wiring together.

****Correction:**** this independence does not hold for minting specifically — see ****[L-05]****. `mint()`'s final step calls the state-changing `Operations2.removeCanMint`, which is itself `whenNotPaused` on Operations2's own flag, so pausing only Operations2 does silently break `Cngn3.mint()` even though Cngn3 itself remains unpaused.

Status

Acknowledged

[Q-05] Operations2 - `onlyOwnerOrTrustedContract` would be bypassable if the Forwarder were ever added as a trusted contract

Description

`Admin2` does not implement ERC-2771 (no `_msgSender()` override) and never strips the 20-byte address the `Forwarder` appends to `calldata`. `onlyOwnerOrTrustedContract` authorizes any address in `trustedContract`. Under the current configuration only `Cngn3` is expected to be trusted, and `Cngn3` never calls into `Admin2`'s minter/blacklist administration functions, so this is closed today. However, if the `Forwarder` contract's address were ever also registered via `addTrustedContract`, a `canForward`-whitelisted signer could route `req.to = Admin2`, `req.data = addCanMint(attacker)` through the `Forwarder` — `Admin2` would see `msg.sender == Forwarder` (a trusted contract) and never learn which end-user actually authorized the call, bypassing `onlyOwner`. Purely a configuration risk under the current deployment; flagged so `addTrustedContract` is never pointed at the `Forwarder` without adding sender-stripping logic to `Admin2` first.

Status

Acknowledged

[Q-06] Operations2#addMintAmount - Message-less boolean-constant `require`, previously flagged and marked "Resolved" by a prior audit but still present

Description

```
function addMintAmount(address _User, uint256 _Amount) public whenNotPaused onlyOwner
returns (bool) {
    require(canMint[_User] == true);    // no revert message; boolean-constant
comparison
    mintAmount[_User] = _Amount;
    ...
}
```

This is the only `require` in the three in-scope files with no revert-reason string, and it uses a `== true` boolean comparison instead of the bare variable. CertiK's earlier report flagged this exact line (as CSU-15 "Comparison to Boolean Constant" and CSU-17 "Missing Error Message") and recorded it as "Resolved" via specific commit hashes — but the current `Operations2.sol` still contains the identical pattern. Purely cosmetic/informational (a failed `require` here still reverts, just without a helpful message), but worth flagging that the previously-claimed fix does not appear to have carried through to this version, in case the same drift happened to other, higher-severity "Resolved" items that fell outside this review's ability to independently verify (e.g. items scoped only to `Multisig.sol`, which no longer exists in this codebase and could not be checked).

Status

Resolved

[Q-07] Confirm multisig custody continuity from the previously-audited deployment

Description

CertiK's reports describe transferring `owner()` of `Operations2/Cngn2` to a 2-of-2 Safe multisig (verified by CertiK on Base Sepolia testnet) as the accepted mitigation for the centralization risk inherent in `onlyOwner` controlling pause, blacklist, mint authorization, and forwarder/bridge configuration. That custody arrangement cannot be verified from source — it depends on which address actually holds `owner()` on the current mainnet deployments of `Cngn3/Operations2/Forwarder` (different contract addresses than what CertiK reviewed).

Recommend confirming that the same or an equivalent multisig custody was carried over to every live deployment listed in the README, rather than assuming it by inheritance from the prior audit.

Note

Client's comment: Tracked separately — outside source-code scope

Status

Acknowledged

[Q-08] Cngn3#transfer/transferFrom - Redemption burn is indistinguishable from a normal transfer on-chain

Description

When the redemption branch fires (external-whitelisted sender → internal-whitelisted recipient), Cngn3 transfers the tokens and then burns them from the recipient in the same call, but only the standard `Transfer(from, to, amount)` event is emitted for the transfer leg (the subsequent `_burn` emits its own `Transfer(to, address(0), amount)`, but nothing ties the two together or flags this as a redemption). Off-chain tooling, indexers, and block explorers watching for a simple balance-credit `Transfer` event cannot distinguish "recipient now holds these tokens" from "recipient held these tokens for one instant before they were destroyed."

This exact issue was raised against an earlier version of this same redemption pattern by a prior audit (FailSafe, High severity: "External-to-Internal Redemption Flow Results in Unintended Token Destruction"), and the team's response at the time was that this is intentional bridge mint/burn architecture — accepted as "Acknowledged" rather than fixed. The current `Cngn3.sol` carries the identical logic in both `transfer()` and `transferFrom()`, with no dedicated `Redemption/Burned` event ever added to flag this path distinctly. A victim whose tokens are force-routed through this branch via someone else's allowance sees only ordinary `Transfer` events in their history, with nothing marking the destruction as involuntary.

Recommendation

Emit a dedicated `Redemption(address indexed from, address indexed to, uint256 amount)` event whenever the auto-burn branch fires, separate from the standard ERC20 `Transfer` events.

Status

Acknowledged

[Q-09] Cngn3#destroyBlackFunds - Burning a blacklisted user's balance does not clear their outstanding allowances

Description

`destroyBlackFunds` reduces `_balances[_blackListedUser]` and `_totalSupply` but never touches `_allowances[_blackListedUser][spender]` for any `spender`. Assessed here as QA rather than a scored Low: `_transfer/_spendAllowance` always check `fromBalance >= amount` before moving funds, so any spender holding a stale allowance against the now-zero (or reduced) balance cannot move more than what remains — there is no path for a stale allowance alone to cause fund loss once the balance backing it is gone. Optionally zero the allowances during `destroyBlackFunds` for cleanliness/documentation purposes, but this is not exploitable as-is.

Status

Acknowledged

[Q-10] Forwarder rotation could enable `_msgSender()` spoofing if `trustedForwarderContract` is ever pointed at a malicious contract

Description

`Cngn3._msgSender()` trusts any contract at `trustedForwarderContract` to correctly append the real signer's address; if `updateForwarderContract` (owner-only) is ever pointed at a malicious or buggy contract, that contract could forge `_msgSender()` for arbitrary addresses, bypassing whitelist/blacklist checks and impersonating any user in `mint/burnByUser/transfer` flows. Assessed here as QA/informational rather than a scored High: this requires the owner to make a privileged configuration change to a malicious target, which is an admin-compromise/misconfiguration scenario (consistent with "admin privileges by design" not being separately scored) rather than an attacker-triggerable path — the code today validates the new address is non-zero but cannot validate its behavior. Recommendation: require the new forwarder to be validated (e.g., a fixed/audited implementation deployed via a known factory), and consider a multisig/timelock for `updateForwarderContract` calls, matching the custody recommendation in **[Q-07]**. Migrating to OpenZeppelin's standard `ERC2771ContextUpgradeable` would also reduce the custom-code surface here.

Status

Acknowledged

[Q-11] Cngn3#destroyBlackFunds - Centralized fund-confiscation power is inherent to the design

Description

The owner can destroy the entire balance of any address `adminOperationsContract` reports as blacklisted. This is documented, intentional functionality (the README lists "Blacklisting capabilities for compliance requirements" as a core feature) rather than a code defect, so it is not scored as a vulnerability here — but it is a real centralization/confiscation risk worth governance controls: consider requiring a timelock or multisig approval for `destroyBlackFunds` specifically, given its irreversible, unilateral nature, and ensure the blacklist criteria enforced by `adminOperationsContract` are themselves governed and auditable.

Status

Acknowledged

[Q-12] Operations2 - `removeCanMint/removeCanForward` lack the `notBlacklisted` modifier their `add*` counterparts have

Description

Flagged via inline `@audit` code comments at `Operations2.sol:90` and `Operations2.sol:159`. `addCanMint/addCanForward` both carry `notBlacklisted(_User)` (you cannot grant mint/forward permission to an already-blacklisted address), but `removeCanMint/removeCanForward` do not carry the same modifier. Reviewed and assessed as intentional/non-issue rather than a gap: `notBlacklisted` on a `_revocation_` function would be actively undesirable, since it would make it impossible to revoke mint/forward permission from a user after they become blacklisted — the current (unguarded) behavior is the safer one. No action recommended; documented here for completeness since the inline comments raised it.

Status

Acknowledged

[Q-13] Operations2 - `addTrustedContract/removeTrustedContract` do not validate the target has contract code

Description

Flagged via inline `@audit` code comments at `Operations2.sol:172` and `Operations2.sol:178`. Neither function checks that `_trustedContract` has code deployed (e.g. via `extcodesize`) before granting/revoking `trustedContract` status. Assessed as low-materiality: (1) this is an `onlyOwner`-gated action, so it only matters if the owner makes a mistake, not an attacker-triggerable path; (2) a code-size check is unreliable as a security control in Solidity (a contract has zero code size during its own constructor, so this check can be trivially bypassed by a contract calling out mid-construction, and conversely there's no security reason an EOA `_couldn't_` validly hold this flag — the privilege is purely `msg.sender`-based). Optional: add an off-chain/deploy-script sanity check rather than an on-chain `extcodesize` guard, which would give a false sense of security.

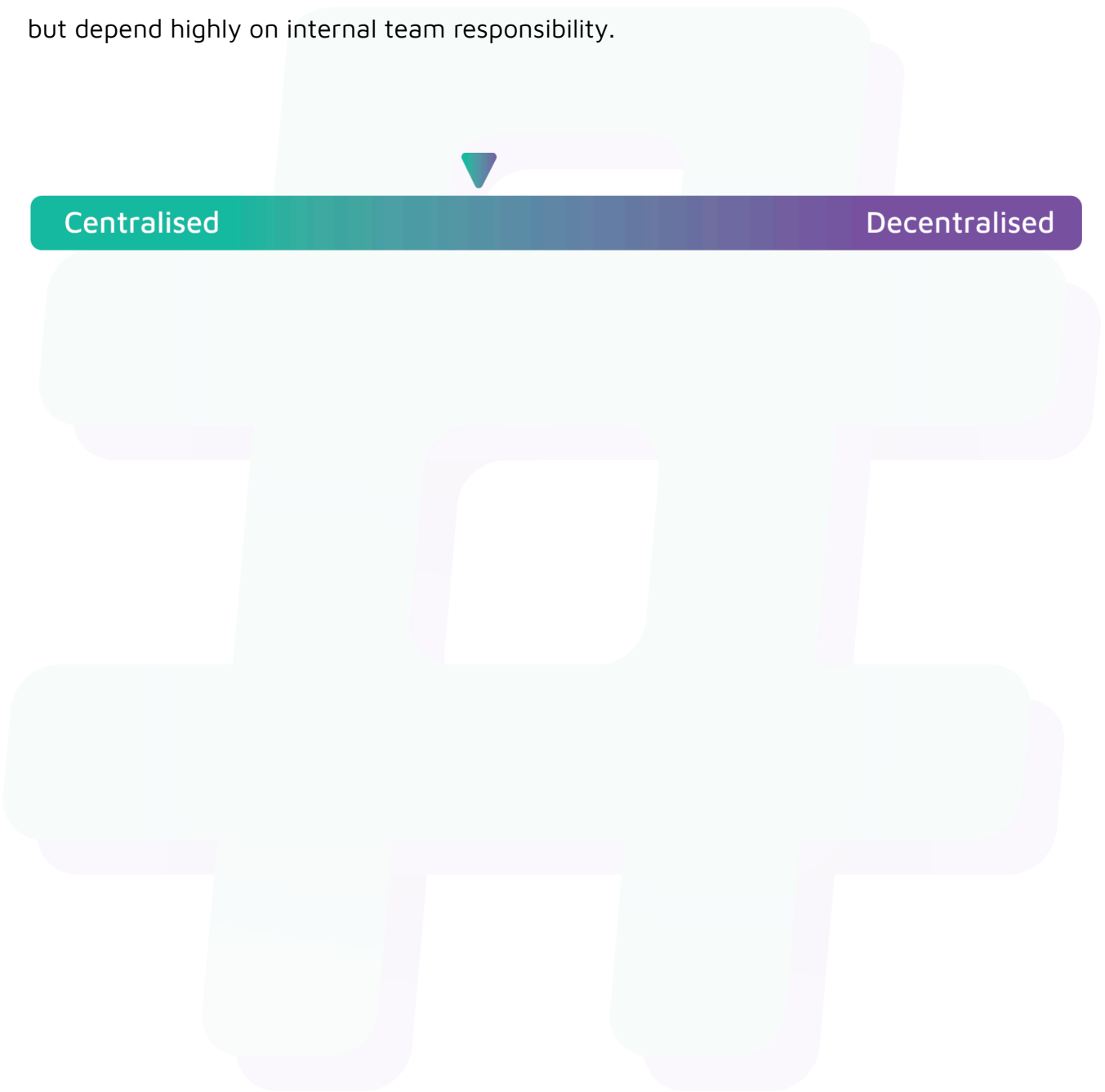
Status

Acknowledged

Centralisation

The cNGN project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the cNGN project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.